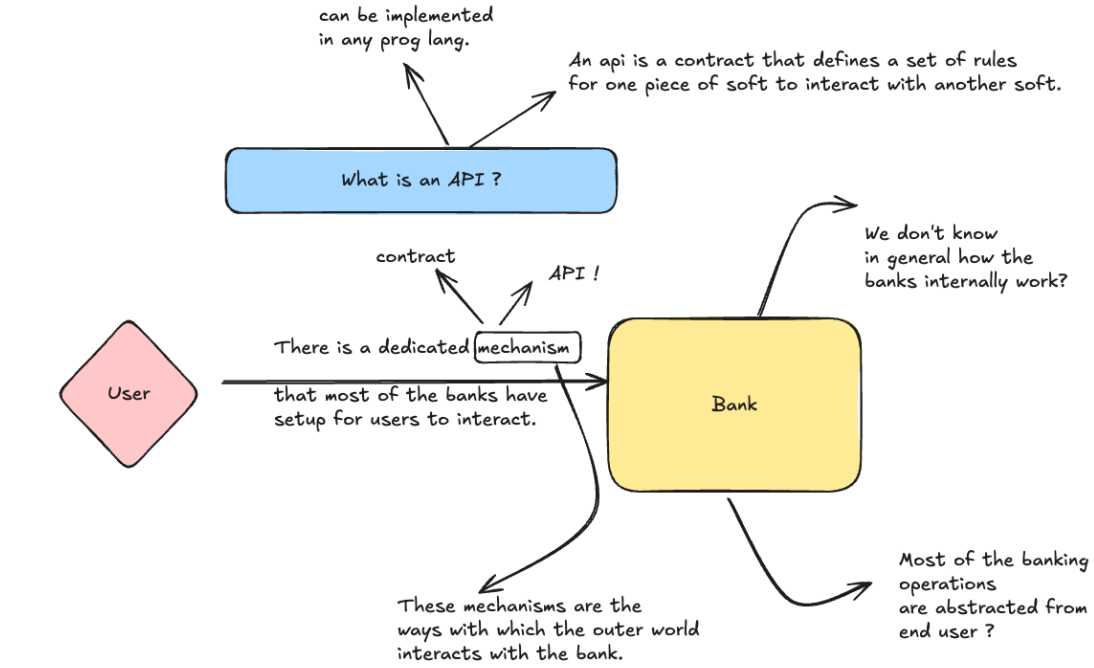
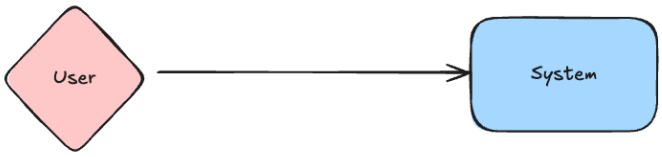
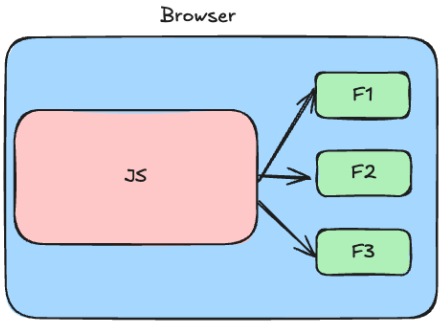




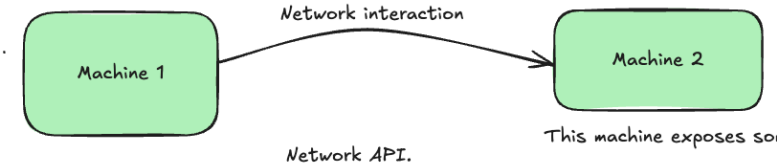
System is going to define a contract !

This contract will be having the details
on how the user with interact with the system.
What input and output the system expects and
gives and more ...

This contract is what we define API as.



setTimeout(fn, t) ⟶ Browser API



Network API.

This machine exposes some functionality

This machine should ideally define APIs (contract
on how the interaction will be done and input/output
info).

How to implement Network APIs ?

There are recommendations to implement APIs. If we won't follow these, then nothing will break unless there is algorithmic in our code. But it's just that, people might find reading your APIs a bit, if it doesn't follow a set recommendation.

Technically, not following the recommendation can impact the code readability.

1. SOAP
2. REST
3. GraphQL
4. gRPC

and more ..

VVI.

Most of the above API recommendations define how to send the data between systems, what format we can use to send the data, how should the contract look like and so on.

REST APIs

REST stands for Representational state transfer.

These are recommendation to create APIs.

Principle and ideas on which REST API recommendation are decided:

1. Platform independence

2. Loose coupling

To implement REST APIs we can use any prog lang, but our implementation should be done in a way that the final API contract follows the below recommendation:

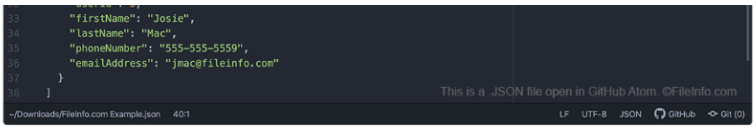
1. For any data exchange (payload exchange), we should use JSONs.

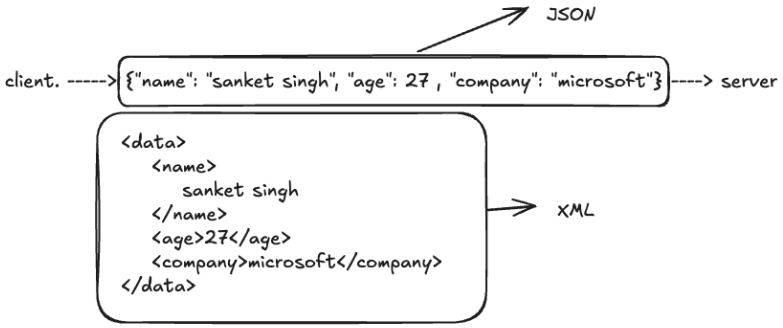
What is JSON ? Javascript object notation. It has nothing to do with JS. It's just that data written in the form of a JSON looks like a JS object. It is a bunch of key value pairs, which describe the data. A JSON file has an extension .json .

FileInfo.com Example.json — ~/Downloads

FileInfo.com Example.json


```
{
  "users": [
    {
      "userId": 1,
      "firstName": "Chris",
      "lastName": "Lee",
      "phoneNumber": "555-555-5555",
      "emailAddress": "clee@fileinfo.com"
    },
    {
      "userId": 2,
      "firstName": "Action",
      "lastName": "Jackson",
      "phoneNumber": "555-555-5556",
      "emailAddress": "ajackson@fileinfo.com"
    },
    {
      "userId": 3,
      "firstName": "Ross",
      "lastName": "Bing",
      "phoneNumber": "555-555-5557",
      "emailAddress": "rbing@fileinfo.com"
    },
    {
      "userId": 4,
      "firstName": "David",
      "lastName": "Reeves",
      "phoneNumber": "555-555-5558",
      "emailAddress": "dreeves@fileinfo.com"
    },
    {
```



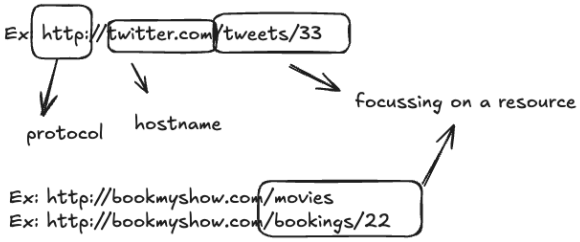


→ 2. Because REST APIs involve network interaction, we have a recommended of defining the URLs of a rest api.

REST API urls are designed around `resources` . What is a resource ? Resource is any kind of data, object or any service representation some real life use case and can be accessed by a client.

For ex: Twitter -> user is a resource, tweet is a resource, like is resource, comment is a resource. etc
signup is not a resource, it is an action.

Resources are nouns, and API urls are designed keeping these nouns in mind not the verbs (actions)



creating a tweet ----> action

in a rest api url is not going to be
/createTweet ✕

→ With the combination of how the URL is written and the HTTP verb/ request method, we can easily identify/guess what the API must be doing

Generally in the urls, we use plural names.

POST request is generally for resource creation

Resource	POST	GET	PUT	DELETE
/customers	Create a new customer	Retrieve all customers	Bulk update of customers	Remove all customers
/customers/1	Error	Retrieve the details for customer 1	Update the details of customer 1 if it exists	Remove customer 1
/customers/1/orders	Create a new order for customer 1	Retrieve all orders for customer 1	Bulk update of orders for customer 1	Remove all orders for customer 1

customer id

All the urls are focussing on the resource

more than one resource can be combined in a url , generally this happens when these resources

/create-customer. ✕
/customers POST

are somewhat related
or associated

/resourceName. + POST --> we are creating a resource
/resourceName + GET --> we are fetching all the records of the resource
/resourceName + DELETE ---> delete all the records of the resource
/resourceName + PUT ---> bulk update
/resourceName/:resourceId. + GET ---> we are fetching a particular single record of the resource identified by the resourceId

colon here represents that this part of the url
is variable and can take any value

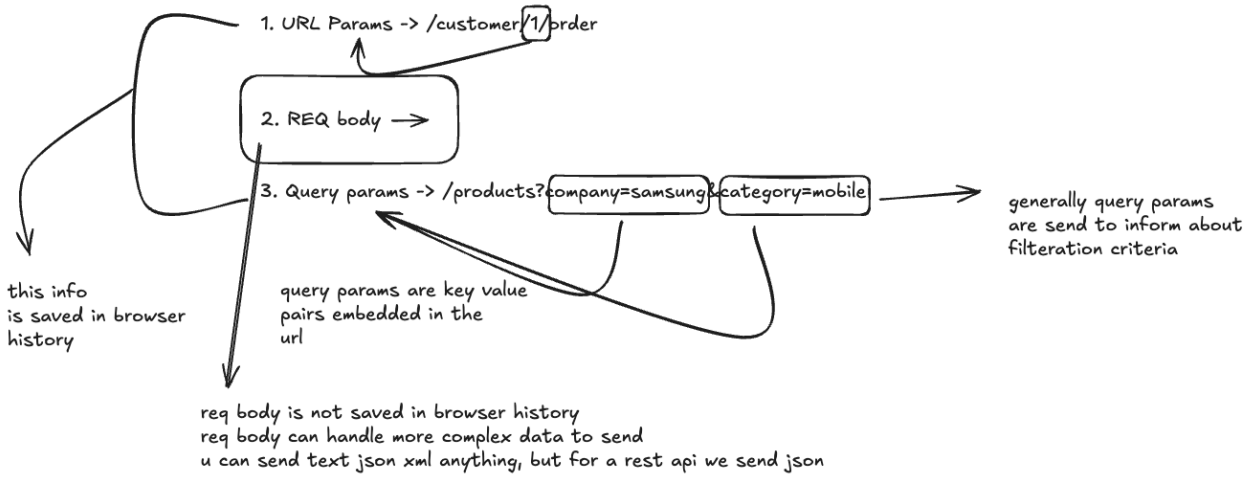
/users/:user_id/bookings + POST ---> we want to create a new booking for a specific user.

/users/:user_id/orders/:order_id + DELETE -> we want to delete order with given order id of the cutomer belonging to the given user_id

→ Response of the REST API should contain a numeric reprensation of what is the status of the api i.e. whether it was successfull failure or what. This number is called as status code.

→ REST APIs should be stateless, it means if multiple http requests are made , then all of those are independent of each other and don't know about each other.

→ To send data, in a REST API, we can use the following three ways:



req body is not saved in browser history
req body can handle more complex data to send
u can send text json xml anything, but for a rest api we send json

→ it is recommended that we version our apis, and version info should be part of the url

/api/v1/products

/api/v3/orders/3/route

this part specifically tells that this is a url of an api + what version of api we are using

another way to send api version is in req headers